

Sprint 4

Índice

Nivel 1-----	2
Ejercicio 1-----	2
Ejercicio 2-----	3
Ejercicio 3-----	5
Ejercicio 4-----	6
Nivel 2-----	7
Ejercicio 1-----	7
Ejercicio 2-----	8
Ejercicio 3-----	9
Ejercicio 4-----	10

Nivel 1

Ejercicio 1

El Country Manager de Alemania necesita revisar urgentemente las transacciones del día 12 de marzo de 2022.

- Escribe la consulta que une (JOIN) transacciones y compañías.
- Filtra los resultados por la fecha indicada y el país "Germany".
- Sin ejecutar la consulta, realiza un "Dry Run" (auditoría de costes).

```
SELECT *  
FROM sprint3_silver.transactions_clean  
JOIN sprint3_silver.companies_clean  
  ON transactions_clean.business_id = companies_clean.company_id  
WHERE country = 'Germany' AND transaction_timestamp = '2022-03-12';
```

✓ Esta consulta procesará 11.86 MB cuando se ejecute.



Fíjate en que BigQuery lee casi toda la tabla a pesar de pedir solo un día (Full Table Scan).

Ejercicio 2

Tras el diagnóstico del ejercicio anterior, hemos confirmado que filtrar la tabla de transacciones por fecha o por empresa provoca un Full Table Scan (lectura completa de la tabla), lo que es ineficiente. Para solucionarlo en la capa de consumo (Gold), materializaremos una versión optimizada de la tabla de hechos.



Al utilizar un entorno gratuito sin tarjeta de crédito, BigQuery borra automáticamente los datos de las particiones con más de 60 días de antigüedad. Dado que nuestros datos históricos son antiguos, si particionamos directamente la tabla, ésta acabará quedando vacía. Para evitarlo, primero actualizaremos automáticamente las fechas de los datos, y después, aplicaremos la optimización física.

- 1 Crea una tabla intermedia llamada *sprint3_silver.transactions_recent* a partir de la tabla *sprint3_silver.transactions_clean*. Tu objetivo es mantener todas las columnas, pero sustituir el timestamp original por uno nuevo, generado aleatoriamente para que se caiga dentro de los últimos 50 días.



Para hacerlo sin escribir todas las columnas a mano, puedes utilizar `SELECT * EXCEPT(timestamp)`. Para calcular la fecha aleatoria, tendrás que restar una cantidad de días al instante actual. Te serán muy útiles las funciones `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, y la combinación `CAST(RAND() * 50 AS INT64)` para generar los días de desfase.

```
1 CREATE TABLE sprint3-analytics-alejandro.sprint3_silver.transactions_recent AS
2 SELECT * EXCEPT(transaction_timestamp), TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST (RAND() * 50 AS INT64) DAY) AS transaction_timestamp
3 FROM sprint3_silver.transactions_clean
```

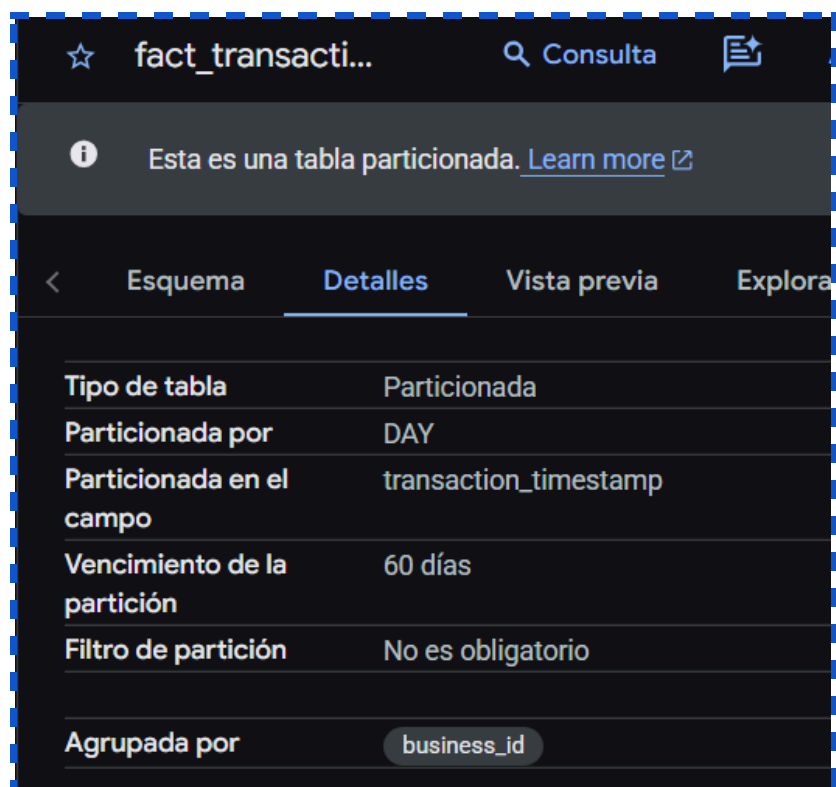
business_id	declined	transaction_timestamp
b-2422	0	2026-05-18 10:59:30.672065 UTC
b-2494	0	2026-05-09 10:59:30.672065 UTC
b-2390	0	2026-05-23 10:59:30.672065 UTC
b-2494	0	2026-05-12 10:59:30.672065 UTC
b-2242	0	2026-05-02 10:59:30.672065 UTC
b-2446	0	2026-04-22 10:59:30.672065 UTC

2 Ahora, crea tu tabla definitiva *sprint3_gold.fact_transactions_optimized* a partir de los datos recientes que acabas de generar en *transactions_recent*. Tienes que construir la sentencia DDL para configurar la tabla con las siguientes estrategias físicas de almacenamiento:

- Particionamiento (Partitioning): Divide la tabla por la fecha del campo DATE(timestamp). Esto permitirá que las consultas que filtren por día (WHERE date = ...) solo lean la partición necesaria.
- Clustering: Ordena los datos dentro de cada partición por business_id. Esto acelerará drásticamente los filtros por compañía y los cruces (JOINS) con la dimensión de empresas.

```
6 CREATE OR REPLACE TABLE sprint3-analytics-alejandro.sprint3_gold.fact_transactions_optimized
7 PARTITION BY DATE(transaction_timestamp)
8 CLUSTER BY business_id AS
9 SELECT * FROM sprint3-analytics-alejandro.sprint3_silver.transactions_recent;
```

Esta declaración creó una nueva tabla con el nombre fact_transactions_optimized. [Ir a la tabla](#)



☆	fact_transacti...	🔍 Consulta	📄
📘	Esta es una tabla particionada. Learn more		
<	Esquema	Detalles	Vista previa
Tipo de tabla	Particionada		
Particionada por	DAY		
Particionada en el campo	transaction_timestamp		
Vencimiento de la partición	60 días		
Filtro de partición	No es obligatorio		
Agrupada por	business_id		

(Captura de pantalla de los "Detalles" de la nueva tabla *fact_transactions_optimized* donde se puede comprobar que está particionada y agrupada en clusters correctamente)

Ejercicio 3

Ya has creado la tabla *sprint3_gold.fact_transactions_optimized* con las estrategias de Particionamiento y Clustering. Ahora toca demostrar en el equipo la mejora de rendimiento y el ahorro de costes que has conseguido. Simularemos una petición típica de negocio: "Queremos ver todas las transacciones de los últimos 30 días".

El objetivo es claro y directo: aplicar exactamente la misma consulta a dos tablas diferentes y comparar su coste computacional.

Construye una consulta SQL que seleccione todas las columnas (SELECT *) y filtre los datos de los últimos 30 días.



NO ejecutes las consultas todavía. En este ejercicio solo utilizaremos el validador de BigQuery ("Dry Run", arriba a la derecha del editor) para visualizar los Bytes procesados de forma gratuita.

Haz la comparativa siguiendo estos dos pasos:

- Paso 1 (Tabla no optimizada): Apunta consulta a *sprint3_silver.transactions_recent*. Observa el validador y realiza una captura de pantalla de los bytes que procesaría.

```
1 SELECT *
2 FROM sprint3_silver.transactions_recent
3 WHERE transaction_timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY)
4 ;
```



Esta consulta procesará 11.85 MB cuando se ejecute.

- » Paso 2 (Tabla optimizada): Sin cambiar nada de la lógica de la consulta, modifica solo el FROM para que ahora apunte a *sprint3_gold.fact_transactions_optimized*. Observa cómo cambia el validador.

```
1 SELECT *
2 FROM sprint3_gold.fact_transactions_optimized
3 WHERE transaction_timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY)
4 ;
```



Esta consulta procesará 6.74 MB cuando se ejecute.

La primera consulta procesa un 75,81% más bytes que la segunda.

En cuanto a bytes facturados, serán 12MB para la primera consulta y 10MB (ya que es el mínimo facturado), lo que significa un ahorro del 16,66%.

Ejercicio 4

El Director General tiene un cuadro de mando que muestra las "Ventas Totales por Día". Él (y otros 50 managers) refrescan este gráfico constantemente.



Si utilizas una consulta normal o una Vista estándar, cada vez que alguien refresca la página, BigQuery recalcula la suma de millones de filas. Es lento y estás pagando por el mismo cálculo una y otra vez.



Utilizar una vista materializada. Esta tecnología guarda el resultado del cálculo en la memoria caché y solo la actualiza si entran nuevos datos (incremental).

- Crea una Vista Materializada llamada `sprint3_gold.mv_daily_sales` que muestre las Ventas Totales por Día.
- Haz una consulta que muestre la vista creada y realiza una captura de pantalla que muestre en su conjunto la vista materializada creada, con la consulta ejecutada y los bits procesados.

```
1 CREATE MATERIALIZED VIEW sprint3_gold.my_daily_sales AS
2 SELECT COUNT(transaction_id) AS transactions_count, SUM(amount) AS total_amount, DATE(transaction_timestamp) AS
   transaction_date
3 FROM sprint3_gold.fact_transactions_optimized
4 GROUP BY transaction_date;
```



Esta declaración creó una nueva vista materializada denominada `my_daily_sales`.

[Ir a la vista materializada](#)

```
1 SELECT * FROM sprint3-analytics-alejandro.
   sprint3_gold.my_daily_sales
```

Se completó la consulta
Se está usando la cuota de procesamiento según demanda

Resultados de... Chat

	Información del trabajo	Resultados	Visualiz
Fila	transactions_count	total_amount	transaction_date
1	968	256679.0799999...	2026-05-27
2	1005	267016.6900000...	2026-04-07
3	1904	489040.3999999...	2026-05-14
4	1924	488748.6000000...	2026-05-25
5	1927	498017.1400000...	2026-05-10
6	1934	501942.5099999...	2026-04-13
7	1935	495291.2800000...	2026-04-23
8	1944	500755.1700000...	2026-05-07
9	1945	512643.2400000...	2026-05-12
10	1955	502455.4100000...	2026-04-11
11	1956	499985.7900000...	2026-04-28
12	1960	519306.2000000...	2026-05-03

Bytes procesados 1.2 KB

Nivel 2

La dirección comienza a hacer preguntas que no se resuelven con un simple GROUP BY. Necesitamos análisis de tendencias y comportamiento de usuario.

Ejercicio 1

El equipo de Marketing quiere analizar el comportamiento de nuestros mejores clientes para diseñar la estrategia del próximo año. Definimos "VIP" como aquellos con un gasto acumulado superior a 500 €. Necesitan un informe que, para cada VIP, muestre su nombre, contacto y su patrón de compra: cuántas veces ha comprado, cuánto gasta de media y cuál fue su compra récord.

- Crea una CTE llamada *VIP_Stats* que agrupe por usuario y calcule:
 - El Gasto Total (SUM).
 - La Cantidad de Transacciones (COUNT).
 - El Ticket Medio (AVG), redondeado a 2 decimales.
 - La Compra Máxima (MAX).
 - Filtro: Mantiene solo aquellos cuyo Gasto Total sea > 500.
- Cruza la CTE con *users_combined* para obtener los datos personales.
 - Columnas: *user_id*, *nombre_completo*, *email*, *num_compras*, *ticket_medio*, *max_compra*, *total_gastado*.
 - Ordenado por *total_gastado* descendente.

```
1 WITH VIP_Stats AS (  
2   SELECT user_id, SUM(amount) AS total, COUNT(transaction_id) AS transaction_count, ROUND(AVG(amount),2) AS transaction_average, MAX(amount) AS max_amount  
3   FROM sprint3_gold.fact_transactions_optimized  
4   GROUP BY user_id  
5   HAVING total > 500  
6 )  
7  
8 SELECT users_combined.user_id, CONCAT(name, ' ', surname) AS nombre_completo, email, transaction_count AS num_compras, transaction_average AS ticket_medio,  
9   max_amount AS max_compra, total AS total_gastado  
10  FROM sprint3_silver.users_combined  
11  JOIN VIP_Stats  
12  ON users_combined.user_id = VIP_Stats.user_id  
   ORDER BY total DESC;
```

✓ Se completó la consulta
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta [Chat](#) [Guardar los resultados](#) [Abrir en](#)

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución		
Fila	user_id	nombre_completo	email	num_compras	ticket_medio	max_compra	total_gastado
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	94	441.35	554.71	41486.54
2	318	Bnyr Astuw	bnyr.astuw@example.com	91	443.39	615.51	40348.50000000...
3	455	Fwfil Phylgmhnd	fwfil.phylgmhnd@example.com	66	522.6	724.44	34491.27000000...
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	78	410.46	675.78	32016.10999999...
5	185	Molly Gilliam	donec@outlook.couk	110	275.75	659.31	30333.03
6	285	Brcxpmj Lnnpa	brcxpmj.lnnpa@example.com	70	417.85	630.52	29249.51000000...

Ejercicio 2

La Dirección Financiera quiere monitorear la "velocidad de ventas" diaria. Necesitan un informe que compare el rendimiento de cada día contra el día anterior para detectar caídas bruscas o picos de crecimiento (Day-over-Day Growth).

Crea una consulta sobre la vista materializada que genere un informe con las siguientes 4 columnas:

- Fecha: La fecha de venta.
- Ventas_Hoy: El total vendido ese día.
- Ventas_Ayer: El total vendido el día anterior (usando funciones de ventana).
- Diff_Percentual: El porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeado a 2 decimales.



Utiliza la función LAG(campo) OVER (ORDER BY data) para leer el valor de la fila anterior sin necesidad de realizar JOINS complejos.

```
1 SELECT
2   transaction_date AS Fecha,
3   ROUND(total_amount,2) AS Ventas_Hoy,
4   ROUND(LAG(total_amount) OVER (ORDER BY transaction_date),2) AS Ventas_Ayer,
5   CONCAT(ROUND((total_amount - LAG(total_amount) OVER (ORDER BY transaction_date)) * 100.0 / LAG(total_amount) OVER (ORDER BY transaction_date), 2), '%')
   AS Diff_Percentual
6 FROM sprint3_gold.mv_daily_sales
7 ORDER BY transaction_date DESC;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta [Chat](#) [Guardar los resultados](#) [Abrir en](#)

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	Fecha	Ventas_Hoy	Ventas_Ayer	Diff_Percentual	
1	2026-05-27	256679.08	536472.84	-52.15%	
2	2026-05-26	536472.84	488748.6	9.76%	
3	2026-05-25	488748.6	520585.26	-6.12%	
4	2026-05-24	520585.26	513302.03	1.42%	
5	2026-05-23	513302.03	499417.56	2.78%	
6	2026-05-22	499417.56	521189.67	-4.18%	
7	2026-05-21	521189.67	515665.8	1.07%	
8	2026-05-20	515665.8	511213.6	0.87%	
9	2026-05-19	511213.6	526527.04	-2.91%	
10	2026-05-18	526527.04	534796.56	-1.55%	

Ejercicio 3

El Director Financiero (CFO) necesita visualizar la curva de crecimiento anual. No le interesa tanto lo que se vendió hoy, sino cuánto llevamos facturado en total desde el inicio del año hasta la fecha (YTD - Year To Date).

Utilizando la vista materializada *mv_daily_sales* (para no recalcular agregaciones base), genera un informe con tres columnas:

- Fecha.
- Ventas del Día: Redondeado a 2 decimales.
- Ventas Acumuladas YTD: Una suma progresiva de las ventas que debe reiniciarse cada 1 de Enero. El resultado final debe estar redondeado a 2 decimales.

?

Para conseguir el reinicio anual, necesitas dividir la ventana usando `PARTITION BY EXTRACT(YEAR FROM data)`. La estructura completa sería: `SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

```
1 SELECT transaction_date AS Fecha,
2        ROUND(total_amount, 2) AS Ventas_del_Dia,
3        ROUND(SUM(total_amount) OVER (
4          PARTITION BY EXTRACT(YEAR FROM transaction_date)
5          ORDER BY transaction_date
6          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
7        ), 2) AS Ventas_Acumuladas_YTD
8 FROM sprint3_gold.mv_daily_sales
9 ORDER BY Fecha;
```

✓ Se completó la consulta
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo		Resultados	Visualización
Fila	Fecha	Ventas_del_Dia	Ventas_Acumula...
1	2026-04-07	267016.69	267016.69
2	2026-04-08	537475.25	804491.94
3	2026-04-09	522282.23	1326774.17
4	2026-04-10	526494.43	1853268.6
5	2026-04-11	502455.41	2355724.01
6	2026-04-12	542690.15	2898414.16
7	2026-04-13	501942.51	3400356.67
8	2026-04-14	515557.02	3915913.69
9	2026-04-15	518218.9	4434132.59

Ejercicio 4

El equipo de Marketing lanza la campaña "A la tercera va la vencida". Quieren enviar un regalo a los usuarios que hayan completado su tercera transacción. Sin embargo, el presupuesto del regalo no es fijo: depende de la "calidad" del cliente en sus inicios. Necesitan saber el Ticket Medio de estas 3 primeras compras para segmentar si envían un detalle básico o un regalo premium.

Genera un listado de los usuarios que han llegado (o superado) su tercera compra, mostrando:

- Datos de usuario (user_id, nombre_completo, email).
- La fecha y el importe exacto de la 3ª compra.
- Media de las 3 primeras: La media de gasto de sus transacciones 1, 2 y 3.

Tienes que utilizar la función ROW_NUMBER() para establecer el orden cronológico de las compras.

?

Tienes que utilizar la cláusula QUALIFY para filtrar.

Reto de Optimización: Busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el historial histórico es costoso. Trate de filtrar las filas necesarias antes de realizar la agregación final.

```
1 WITH Filter_Three AS (  
2     SELECT user_id,  
3           transaction_timestamp,  
4           amount,  
5           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY transaction_timestamp) AS transaction_count  
6     FROM sprint3_gold.fact_transactions_optimized  
7     QUALIFY transaction_count <= 3  
8 ),  
9 Client_Quality AS (  
10    SELECT user_id,  
11          ARRAY_AGG(transaction_timestamp ORDER BY transaction_count DESC LIMIT 1)[OFFSET(0)] AS third_transaction_date,  
12          ARRAY_AGG(amount ORDER BY transaction_count DESC LIMIT 1)[OFFSET(0)] AS third_transaction_amount,  
13          AVG(amount) AS average_amount,  
14    FROM Filter_Three  
15    GROUP BY user_id  
16 )  
17  
18 SELECT users_combined.user_id,  
19       CONCAT(users_combined.name, ' ', users_combined.surname) AS nombre_completo,  
20       users_combined.email,  
21       Client_Quality.third_transaction_date AS fecha_3a_transaccion,  
22       Client_Quality.third_transaction_amount AS cantidad_3a_transaccion,  
23       ROUND(Client_Quality.average_amount, 2) AS media_3_primeras  
24 FROM Client_Quality  
25 JOIN sprint3_silver.users_combined  
26   ON Client_Quality.user_id = users_combined.user_id  
27 ORDER BY media_3_primeras DESC;
```

Se completó la consulta Se está usando la cuota de procesamiento según demanda							
Resultados de la consulta						Chat	Guardar los resultados ▾
Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución	
Fila	user_id	nombre_completo	email	fecha_3a_transaccion	cantidad_3a_tran...	media_3_primeras	
1	3871	Wkgppo Wxcqutnz	wkgppo.wxcqutnz@example.com	2026-04-14 10:59:30.672065 UTC	635.86	570.76	
2	3706	Ogiekj Ijmvxoru	ogiekj.ijmvxoru@example.com	2026-04-21 10:59:30.672065 UTC	533.29	553.33	
3	455	Fwfl Phylgnmhnd	fwfl.phylgnmhnd@example.com	2026-04-11 10:59:30.672065 UTC	268.5	541.91	
4	4181	Kaupal Qgylkhiu	kaupal.qgylkhiu@example.com	2026-05-02 10:59:30.672065 UTC	517.22	528.3	
5	4787	Ybazzz Yegjpzaf	ybazzz.yegjpzaf@example.com	2026-04-14 10:59:30.672065 UTC	382.06	519.13	
6	385	Gvxmvr Xyopdz	gvxmvr.xyopdz@example.com	2026-04-07 10:59:30.672065 UTC	609.56	519.12	